Lab 4: Vibe Coding - Chat Transcript (Flappy Bird)

Candidate SRN: PES1UG24AM106 | **Assigned Project:** Real-Time Flappy Bird Clone (40_flappy-bird)

```
================================================================================
LAB 4: VIBE CODING - PAIR PROGRAMMING CHAT TRANSCRIPT
================================================================================
Candidate SRN       : PES1UG24AM106
Assigned Project    : Real-Time Flappy Bird Clone (40_flappy-bird)
Course / Module     : Software Engineering Lab 4 (Vibe Coding)
Pair Programmer     : Google DeepMind Gemini Assistant
Deliverable         : Deliverable (c) - Complete LLM Chat History
================================================================================

--------------------------------------------------------------------------------
[TURN 1] USER (PES1UG24AM106)
--------------------------------------------------------------------------------
Real-Time Flappy Bird Clone
This project is a terminal-based Flappy Bird clone using Pygame. It
introduces students to interactive game design using object-oriented
principles and real-time graphical rendering.

What's Provided:
A partially working version of a Flappy Bird game with:
- A player-controlled bird affected by gravity, with a flap on key/click
- Scrolling pipes with a randomized gap
- Score display
You are expected to analyze, interact with an AI assistant, and complete/fix
the game to make it fully functional.

Tasks to Complete:
Task 1: Refine Collision Detection
The bird sometimes clips the edge of a pipe without dying, especially as
pipe speed increases. Investigate and enhance collision accuracy so hits are
detected consistently.

Task 2: Implement Game Over Condition
Add a screen that displays the final score once the bird hits the ground,
ceiling, or a pipe, then gracefully waits for input instead of just printing
to the console.

Task 3: Add Replay Option
```

After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard pipe speed/gap), or exit.

Task 4: Add Sound Feedback
Add basic sound effects for flapping, passing a pipe (scoring), and dying.

Expected Behavior:
- Bird flaps upward on Space or mouse click, and falls due to gravity otherwise
- Pipes spawn at a regular interval and scroll from right to left with a randomized gap
- Score increases by one each time the bird passes a pipe
- Game ends when the bird hits the ground, the ceiling, or a pipe

Repo URL: https://github.com/SETAPESU26/40_flappy-bird/tree/main

Deliverables:
- A 10-second video of gameplay before your changes, showing the bug/broken behavior
- A 10-second video of gameplay after your changes, showing the bug fixed and the new features working
- The Chat/LLM used page link, with the complete chat history

I want you to analyze the codebase, explain how to run it to record the first video, suggest improvements, and provide updated code fulfilling all 4 tasks.

[TURN 2] AI PAIR PROGRAMMER

Let's analyze the Flappy Bird codebase, identify the core bugs, and implement the complete solution.

1. Codebase Analysis & Bug Diagnosis
* Task 1 (Collision Detection): In the original code, collision is checked with:
    pipe.top_rect().collidepoint(self.bird.center())
  This only tests the 1-pixel center point of the bird against the pipe rectangles, completely ignoring the bird's 15-pixel radius! The bird visually clips up to 14 pixels into the edge or corner of any pipe without dying.
  Fix: Implement precise Circle-to-AABB (Axis-Aligned Bounding Box) collision detection:
    closest_x = max(rect.left, min(bird.x, rect.right))
    closest_y = max(rect.top, min(bird.y, rect.bottom))
    dx = bird.x - closest_x; dy = bird.y - closest_y
    is_colliding = (dx*dx + dy*dy) <= (radius * radius)

* Task 2 (Game Over Condition): The original code only prints "Game over!

```
Final score:" to the terminal console and abruptly freezes the window.
  Fix: Render an in-game modal overlay card displaying "GAME OVER", Final
Score, and Best Session Score.

* Task 3 (Replay & Difficulty Selection): Add state resetting with
selectable difficulty presets:
  - Easy: Pipe speed 3, gap 185px, interval 105 frames
  - Medium: Pipe speed 4, gap 150px, interval 90 frames
  - Hard: Pipe speed 6, gap 125px, interval 70 frames
  Keys: [1] Easy, [2] Medium, [3] Hard, [Space/R] Replay, [Q] Quit.

* Task 4 (Sound Feedback): Generate retro 16-bit PCM WAV audio effects using
Python's standard library `wave` and `math`:
  - flap.wav: Upward jump frequency chirp
  - score.wav: Bright two-tone chime
  - hit.wav: Low punch/crash thud
  Managed via `SoundManager` with fallback handling so the game never
crashes if sound drivers are absent.

### 2. How to Record the First Video (Before Changes)
Run:
  python main.py
Demonstrate in 10 seconds:
1. Bird grazing pipe corners/edges without dying due to single-point
collision checking.
2. Complete silence (no audio on flap, score, or death).
3. Window freezing on impact with output printed only to the terminal.

### 3. Implementation Code:
- game/bird.py: Implemented `collides_with_rect(rect)` with exact circle-to-
box distance math.
- game/pipe.py: Implemented proper gaps, top and bottom bounding boxes, and
collar rendering.
- game/sound_manager.py: Audio management for jump, score, and hit sound
effects.
- game/game_engine.py: State machine, collision handling, difficulty
scaling, and game over overlay.
- main.py: Application loop and rendering pipeline.

--------------------------------------------------------------------------
----
[TURN 3] USER (PES1UG24AM106)
--------------------------------------------------------------------------
----
Can you change the updated version in a way like:
1. Once you run the command the game should open and then you should press
start in order to start the game.
2. There should be an option to maximize the screen.

--------------------------------------------------------------------------
----
```

```
[TURN 4] AI PAIR PROGRAMMER
--------------------------------------------------------------------------------
Both enhancements have been integrated into the Flappy Bird project:

1. Interactive Start Screen (game/game_engine.py):
   - Added `STATE_START` so the game does not drop immediately into play.
   - The bird performs an idle hover bobbing animation (using a sine wave)
while waiting.
   - Includes a clickable `[ START GAME ]` button, or press Space/Enter to
begin.
   - Pre-game difficulty buttons: [1] Easy, [2] Medium, [3] Hard.
   - Student candidate watermark: SRN: PES1UG24AM106 displayed on HUD, start
screen, and game over card.

2. Screen Maximize & Resizing Support (main.py):
   - Configured `pygame.display.set_mode((500, 700), pygame.RESIZABLE)`,
enabling the standard Windows Maximize button on the title bar.
   - Smooth scaling via `pygame.transform.smoothscale` preserving the
500x700 aspect ratio with clean black letterboxing borders.
   - Mouse coordinate translation mapping window pixels back to the virtual
canvas so clicking START GAME and buttons works accurately in maximized
mode.
   - Keyboard shortcut: Press F11 or Alt+Enter to toggle fullscreen /
maximize.

--------------------------------------------------------------------------------
[TURN 5] USER (PES1UG24AM106)
--------------------------------------------------------------------------------
How to run these same files on a MacBook?

--------------------------------------------------------------------------------
[TURN 6] AI PAIR PROGRAMMER
--------------------------------------------------------------------------------
Here is how to run the Flappy Bird project on macOS:

1. Copy the folder to your Mac (e.g. to Desktop).
2. Open Terminal (Cmd + Space -> Terminal).
3. Verify Python 3:
     python3 --version
4. Create a virtual environment and install dependencies:
     cd ~/Desktop/flappybird-updated
     python3 -m venv venv
     source venv/bin/activate
     pip install pygame
5. Run the game:
     python3 main.py
```

```
6. Screen Recording on Mac:
   - Press Cmd + Shift + 5 to open the native screen recorder.
   - Select the game window and record 10 seconds.
   - Video automatically saves to the Desktop as a .mov file.
7. Mac Controls:
   - Click the green window button to zoom/maximize.
   - Press Fn + F11 to toggle fullscreen.

--------------------------------------------------------------------------------
[TURN 7] USER (PES1UG24AM106)
--------------------------------------------------------------------------------
c. Chat history exported as a doc/pdf - this is one of the deliverables that
I found. How to do it?

--------------------------------------------------------------------------------
[TURN 8] AI PAIR PROGRAMMER
--------------------------------------------------------------------------------
The complete chat history has been formatted and exported specifically for
SRN: PES1UG24AM106:
- PDF Deliverable: Lab4_Chat_History_PES1UG24AM106.pdf
- Document / Text Deliverable: Chat_History_FlappyBird_PES1UG24AM106.txt

Manual Export Instructions:
1. In your browser or chat tool, press Ctrl + P (or Cmd + P on Mac).
2. Set Destination to "Save as PDF".
3. Check "Background graphics" under More Settings.
4. Click Save to your lab submission directory.
================================================================================
END OF TRANSCRIPT - FLAPPY BIRD (PES1UG24AM106)
================================================================================
```